

Chapter Overview

Chapter 18 covers Nginx as a reverse proxy, API gateway, and load balancer for ShopVerse: configuration syntax, upstream blocks, load balancing algorithms, SSL/TLS termination, HTTP/2, rate limiting, caching, WebSocket proxying, Nginx Ingress Controller in Kubernetes, security headers, performance tuning, and observability (access logs, metrics with Prometheus).

Sub-Chapter	Topic
18-01	Nginx Architecture – master process, worker processes, event loop
18-02	Core Config – nginx.conf, server blocks, location blocks
18-03	Reverse Proxy – proxy_pass, upstream, header forwarding
18-04	Load Balancing – round_robin, least_conn, ip_hash, weights
18-05	SSL/TLS Termination – TLSv1.3, HSTS, OCSP stapling
18-06	HTTP/2 – multiplexing, header compression, activation
18-07	Rate Limiting – limit_req_zone, burst, nodelay
18-08	Caching – proxy_cache_path, cache zones, X-Cache-Status
18-09	WebSocket Proxying – Upgrade, Connection headers, timeout
18-10	Nginx Ingress Controller in Kubernetes – annotations, TLS
18-11	Security Headers – CSP, HSTS, X-Frame-Options, CORS
18-12	Performance Tuning – worker_processes, keepalive, gzip
18-13	Observability – access log format, stub_status, Prometheus exporter

Nginx Process Architecture – event-driven, non-blocking I/O

Master Process – reads config; binds to ports; manages workers

nginx -s reload: new workers load new config; old workers drain connections

nginx -s stop / quit: graceful or immediate shutdown

Worker Processes (worker_processes auto = 1 per CPU core)

Each worker runs a single-threaded epoll event loop

Non-blocking I/O: handles thousands of connections per worker thread

Cache Manager – purges expired objects from disk cache

Cache Loader – loads cache metadata into memory on startup

	Apache httpd (prefork)	Nginx
Concurrency model	Thread per connection	Event loop; 1 thread per worker
Memory per connection	~1 MB (thread stack)	~few KB (event state)
C10K (10k concurrent)	Struggles; thread explosion	Handles easily; kernel epoll
Config hot reload	Full restart required	nginx -s reload – zero downtime

```
user nginx;
worker_processes auto; # 1 per CPU core
worker_rlimit_nofile 65535; # raise open file limit per worker
error_log /var/log/nginx/error.log warn;
events {
worker_connections 4096; # max connections per worker
use epoll; # Linux epoll (most efficient)
multi_accept on; # accept all pending at once
}
http {
include mime.types;
sendfile on; # zero-copy file transfer via kernel
tcp_nopush on; # batch send headers + first data
tcp_nodelay on; # disable Nagle for keepalive
keepalive_timeout 65;
include /etc/nginx/conf.d/*.conf;
}
```

Directive	Value	Effect
worker_processes	auto	1 worker per CPU core; maximises parallelism
worker_connections	4096	Total = workers × 4096 concurrent connections
sendfile	on	Zero-copy via OS; faster static file serving
keepalive_timeout	65s	TCP connection reused for 65s; saves handshake overhead

```
upstream product_service {
server product-service:8080;
keepalive 32; # pool of 32 persistent connections to upstream
}
server {
listen 80; server_name api.shopverse.com;
location /api/v1/products {
proxy_pass http://product_service;
proxy_http_version 1.1;
proxy_set_header Connection ""; # enable upstream keepalive
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_connect_timeout 5s;
proxy_read_timeout 30s;
proxy_next_upstream error timeout;
}
}
```

Header Forwarded	Value	Purpose
X-Real-IP	\$remote_addr	Client IP; Spring reads for geo-IP / rate limit
X-Forwarded-For	\$proxy_add_x_forwarded_for	Full client IP chain through multiple proxies
X-Forwarded-Proto	\$scheme	"https" so Spring generates correct redirect URLs
Host	\$host	Preserve original host for virtual hosting

Algorithm	Directive	Best For
Round Robin	(default; no directive)	Equal-capacity pods; simple distribution
Least Connections	least_conn;	Varying request duration; avoids hot spots
IP Hash	ip_hash;	Session affinity; same client → same server
Weighted	server s:8080 weight=3;	Servers with different capacity
Random two choices	random two least_conn;	Reduces coordination; better than least_conn at high concurrency

```

upstream product_service {
    least_conn;
    server product-svc-1:8080 max_fails=3 fail_timeout=30s;
    server product-svc-2:8080 max_fails=3 fail_timeout=30s;
    server product-svc-3:8080 max_fails=3 fail_timeout=30s;
    keepalive 32;
}
# max_fails=3: mark server down after 3 consecutive failures
# fail_timeout=30s: retry after 30s
# keepalive 32: pool of 32 idle upstream connections per worker

```

```
server {  
    listen 443 ssl http2;  
    server_name api.shopverse.com;  
    ssl_certificate /etc/ssl/certs/shopverse.crt;  
    ssl_certificate_key /etc/ssl/private/shopverse.key;  
    ssl_protocols TLSv1.2 TLSv1.3;  
    ssl_ciphers HIGH:!aNULL:!MD5;  
    ssl_session_cache shared:SSL:10m; # share TLS sessions across workers  
    ssl_session_timeout 1d;  
    ssl_stapling on; # OCSP stapling (no client round-trip)  
    ssl_stapling_verify on;  
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;  
}  
# HTTP -> HTTPS redirect  
server { listen 80; server_name api.shopverse.com;  
    return 301 https://$host$request_uri; }
```

TLS Setting	Value	Purpose
ssl_protocols	TLSv1.2 TLSv1.3	Disable TLS 1.0/1.1 (deprecated; vulnerable)
ssl_session_cache	shared:SSL:10m	Shared across workers; avoids full handshake on reconnect
ssl_stapling	on	Nginx caches OCSP response; client skips OCSP lookup
HSTS	max-age=31536000	Browser uses HTTPS for 1 year; resists SSL stripping

Feature	HTTP/1.1	HTTP/2
Multiplexing	One request per TCP connection	Multiple concurrent streams over one TCP
Header compression	Plain text repeated per request	HPACK; headers delta-encoded
HOL blocking	At HTTP layer (pipelining)	Eliminated at HTTP; persists at TCP
Activation	listen 443 ssl;	listen 443 ssl http2;

Rate Limiting

```

http {
    # 10 MB zone; 10 requests/second per IP
    limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;
}
server {
    location /api/v1/ {
        limit_req zone=api_limit burst=20 nodelay;
        # burst=20: allow 20 extra requests instantly
        # nodelay: serve burst immediately (not queued with delay)
        limit_req_status 429; # 429 Too Many Requests
        proxy_pass http://product_service;
    }
}

```

```

proxy_cache_path /var/cache/nginx
levels=1:2
keys_zone=product_cache:10m
max_size=1g
inactive=60m
use_temp_path=off;
location /api/v1/categories {
proxy_cache product_cache;
proxy_cache_key "$request_uri$http_accept_language";
proxy_cache_valid 200 60m;
proxy_cache_valid 404 1m;
proxy_cache_use_stale error timeout updating;
add_header X-Cache-Status $upstream_cache_status;
proxy_pass http://product_service;
}

```

\$upstream_cache_status	Meaning
HIT	Served from Nginx cache; no upstream call
MISS	Cache miss; upstream called; response now cached
EXPIRED	Cached entry expired; upstream called to refresh
STALE	Stale copy served while background refresh runs
BYPASS	Cache bypassed (no-cache header or pragma)


```
map $http_upgrade $connection_upgrade {  
  default upgrade;  
  '' close;  
}  
location /ws/ {  
  proxy_pass http://websocket_backend;  
  proxy_http_version 1.1; # HTTP/2 lacks Upgrade  
  proxy_set_header Upgrade $http_upgrade;  
  proxy_set_header Connection $connection_upgrade;  
  proxy_read_timeout 3600s; # keep WS alive 1 hour  
  proxy_send_timeout 3600s;  
}
```

Requirement	Reason
proxy_http_version 1.1	HTTP/2 does not support the Upgrade mechanism
Upgrade: websocket header	Signal protocol switch from HTTP to WebSocket
Connection: upgrade	Hop-by-hop header to upgrade the connection
proxy_read_timeout 3600s	WebSocket connections are long-lived; default 60s kills them

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shopverse-ingress
  namespace: shopverse
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/proxy-body-size: "10m"
    nginx.ingress.kubernetes.io/limit-rps: "50"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "30"
    nginx.ingress.kubernetes.io/enable-cors: "true"
spec:
  ingressClassName: nginx
  tls: [{ hosts: [api.shopverse.com], secretName: shopverse-tls }]
  rules:
    - host: api.shopverse.com
      http:
        paths:
          - path: /api/v1/products
            pathType: Prefix
          backend: { service: { name: product-service, port: { number: 8080 } } }
          - path: /api/v1/orders
            pathType: Prefix
          backend: { service: { name: order-service, port: { number: 8081 } } }
```

```
# Add to server {} block
add_header X-Frame-Options "DENY" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-XSS-Protection "1; mode=block" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Content-Security-Policy
"default-src 'self'; script-src 'self';
img-src 'self' https://cdn.shopverse.com; frame-ancestors 'none'" always;
```

Header	Value	Protects Against
X-Frame-Options	DENY	Clickjacking (embedding in iframe)
X-Content-Type-Options	nosniff	MIME-sniffing (e.g. script disguised as image)
Content-Security-Policy	Allowlist	XSS (restrict script/style/img sources)
HSTS	max-age=31536000; includeSubDomains	SSL stripping, protocol downgrade
Referrer-Policy	strict-origin-when-cross-origin	Leaking URL to third-party in Referer header

```

http {
# Gzip compression
gzip on;
gzip_comp_level 6; # 1(fast/low) - 9(slow/best); 6 is balanced
gzip_min_length 1024; # skip tiny responses; overhead not worth it
gzip_types text/plain text/css application/json application/javascript;
# Open file cache - cache file descriptors + stat info
open_file_cache max=10000 inactive=30s;
open_file_cache_valid 60s;
open_file_cache_min_uses 2;
open_file_cache_errors on;
}

```

Tuning Lever	Setting	Value	Effect
Workers	worker_processes	auto	1 per core; max CPU use
Connections	worker_connections	4096	4096 × cores total
Upstream keepalive	keepalive	32 per upstream	Reuse TCP to backend; avoid handshake
gzip	gzip_comp_level	6	Good compression/speed balance
File cache	open_file_cache	max=10000	Avoid open() syscall per static file request

```
# Custom JSON access log format
log_format json_combined escape=json
'{ "time": "$time_iso8601",'
'"remote_addr": "$remote_addr",'
'"method": "$request_method",'
'"uri": "$uri",'
'"status": "$status",'
'"bytes_sent": $bytes_sent,'
'"request_time": $request_time,'
'"upstream_response_time": "$upstream_response_time",'
'"user_agent": "$http_user_agent" }';
access_log /var/log/nginx/access.log json_combined buffer=16k flush=5s;
# Built-in stub_status (for Prometheus nginx-exporter)
server {
    listen 8090; # internal metrics port
    location /nginx_status {
        stub_status;
        allow 10.0.0.0/8; # allow only internal monitoring
        deny all;
    }
}
```

stub_status metric	Meaning
Active connections	Current connections being processed
Requests	Requests per second (calculated by exporter)
Reading/Writing/Waiting	Reading headers / Writing response / Keepalive idle

■ Do

- ✓ Set `worker_processes auto` – one per CPU core; each runs independent epoll event loop.
- ✓ Enable HTTP/2 (`listen 443 ssl http2`) – multiplexing reduces per-request connection overhead.
- ✓ Use `upstream keepalive 32` – reuse connections to Spring Boot backends; saves TCP handshake.
- ✓ Terminate TLS at Nginx; communicate plain HTTP internally – centralises certificate management.
- ✓ Add rate limiting (`limit_req_zone`) before requests reach services – protect against abuse.
- ✓ Use `proxy_cache` for stable read endpoints (categories, config) – reduces backend load significantly.
- ✓ Add all security headers (CSP, HSTS, X-Frame-Options) with `always` flag.
- ✓ Use `nginx -s reload` for config updates – zero-downtime; in-flight requests complete gracefully.
- ✓ Log in JSON format with `upstream_response_time` – enables ELK/Loki ingestion without parsing.
- ✓ Expose `stub_status` on internal-only port for Prometheus exporter.

■ Anti-Patterns

- `worker_processes 1` on multi-core – single core bottleneck under heavy load.
- No `keepalive` to upstream – new TCP connection per request; latency and CPU overhead at scale.
- Exposing backend ports directly (no Nginx) – no rate limiting, no TLS, no security headers.
- HTTP in production – credentials and tokens in plain text; TLS mandatory.
- Caching POST/PUT/DELETE – these mutate state; only cache GET/HEAD responses.
- No `proxy_read_timeout` for WebSocket – default 60s closes long-lived WS connections.
- No security headers – XSS, clickjacking, MIME-sniffing vulnerabilities in browser.

Sub-Chapter	Key Point	One-Liner
18-01 Architecture	Master + workers; epoll	1 worker per core; reload = zero downtime
18-02 Config	worker_connections 4096	sendfile on; keepalive_timeout 65s
18-03 Proxy	proxy_pass + headers	X-Real-IP, X-Forwarded-For, X-Forwarded-Proto
18-04 Load Balancing	least_conn upstream	max_fails=3 fail_timeout=30s; keepalive 32
18-05 TLS	TLSv1.2/1.3 + HSTS	ssl_session_cache; OCSP stapling; 301 http→https
18-06 HTTP/2	listen 443 ssl http2	Multiplexing; no HOL at HTTP layer
18-07 Rate Limiting	limit_req_zone 10r/s	burst=20 nodelay; 429 on exhaustion
18-08 Cache	proxy_cache_path + keys_zone	no_cache; stale on error; X-Cache-Status
18-09 WebSocket	proxy_http_version 1.1	Upgrade+Connection headers; timeout 3600s
18-10 K8s Ingress	annotations + ingressClass	shared; limit-rps; path-based routing
18-11 Security	CSP + HSTS + X-Frame	add_header ... always; frame-ancestors none
18-12 Perf	gzip level 6; open_file_cache	1k min_length; 10k cached file descriptors
18-13 Observability	JSON access log + stub_status	stream_response_time; Prometheus exporter

Interview Quick-Check

Question	Concise Answer
Nginx vs Apache concurrency?	Nginx: event loop per worker; non-blocking I/O; thousands of connections per thread. Apache prefork
How does Nginx rate limiting work?	limit_req_zone defines shared-memory zone with rate. limit_req applies to location with burst. Request
SSL termination at Nginx?	Client → HTTPS → Nginx (decrypts) → HTTP → backend. Nginx holds certificate. Backend is plain
HTTP/2 benefit?	Multiplexing: multiple requests over one TCP connection. HPACK header compression. No head-of-
WebSocket proxy requirements?	Must use HTTP/1.1 (HTTP/2 lacks Upgrade). Forward Upgrade and Connection headers. Set proxy_